# Sound Biosynthetic Design over Iterative Grammars: A Realizability Geometry for Fungal Polyketides

Brage Eilertsen
brageei@uio.no
*University of Oslo*

May 2026

**Abstract**

Predicting a fungal natural product from its biosynthetic gene cluster (BGC) is usually posed as direct BGC $\not\to$structure regression, and it stalls—reported balanced accuracies of 51–68%, barely moved when a thousand bacterial BGCs are pooled with the few hundred fungal ones. We argue the regression targets the wrong object. A product is the image $y = \mathrm{Exec}_\Theta(z; G)$ of a latent biosynthetic *program* $z$ executed under an operator library $\Theta$ and a domain-derived grammar $G$, so the field's ceiling is a statement about grammar *identifiability*—how much of $z$ the observables pin down—not about how much data exists. We build a sound inverse compiler over this program space and run it in three directions. *Forward*, a deterministic executor renders a program to a structure; *inverse*, a beam verifier returns the exact set $Z^\star$ of legal programs consistent with the observables, as a typed verdict—VERIFIED, UNDER-OBSERVED (naming the next observable that would collapse $Z^\star$), or OUT-OF-GRAMMAR. Run inverse across all 326 fungal PKS in MIBiG 4.0, exact reconstruction reaches 8/326—not a disappointment but the *boundary* of the natural manifold the engine operates over (a tailoring-latent reformulation recovers 64% conditional on a producible core)—and a cross-taxa transfer probe locates its one hard axis: per-cycle reduction chemistry transfers at 1.00 within a fixed domain set but collapses to 0.567 across the iteration program—bacterial modular and fungal iterative PKS thus instantiate two computational regimes of catalytic control: *externalized* per-step state (one module per step, learnable) versus *compressed*, state-dependent routing through one reusable manifold.

Run *backward*, the same engine becomes a design tool. We place a desired molecule on a *realizability geometry* against the natural manifold, decomposed onto two axes that map to different wet-lab capabilities—*structural* edits (domain content; mature bacterial modular-PKS precedent) and *control* edits (the iteration program; the 0.567 frontier)—scored by a literature-curated, super-additive cost that *is* the field's measured engineering difficulty rather than an assumption we supply. Across the 1-edit neighbourhood of the manifold the control axis dominates (69 frontier to 49 engineerable), and an experiment-selection planner ranks the observation that would most collapse $Z^\star$ per unit cost. The same iteration-grammar wall is thus a prediction ceiling read one way and a design frontier read the other—a symmetry we formalise as an *identifiability–controllability duality* (Theorem 1, §10): the verifier's consistent set and the designer's specification space are one equivalence class, so prediction and design ambiguity are projections of a single observability quantity $\kappa_O$, a result that travels beyond fungal PKS with a falsifiable prediction on RiPP biosynthesis. Because structure elucidation in this field proceeds by heterologous expression and NMR with mass spectrometry as triage—the methodology of all seven verified products we curate from primary sources—we build a metabologenomics *triage* tool whose three-state verdict names explicitly the clusters where triage suffices and those that must go to expression+NMR. The observability and verdict layers are family-agnostic, demonstrated unchanged over PKS, a non-ribosomal peptide (NRPS) grammar, and a typed PKS–NRPS hybrid; only the program generator is family-specific. All tools, data, and analyses are open and reproducible from single commands.

# 1 Introduction

Many fungal drugs—the cholesterol-lowering agent lovastatin, the antifungal griseofulvin—are polyketides: large carbon scaffolds assembled by a multidomain enzymatic factory, the polyketide synthase (PKS), encoded by a contiguous biosynthetic gene cluster (BGC). Genome sequencing has revealed that filamentous fungi carry vast numbers of such clusters, most of them *cryptic*: no metabolite has ever been linked to them. They are widely regarded as one of the most promising untapped reservoirs of new bioactive chemistry.

Turning a cryptic fungal BGC into a candidate molecule requires answering: *what* would the cluster make? Recent machine learning has attacked this with limited success. Riedling et al. (2024) trained classical classifiers to predict the bioactivity class of fungal secondary metabolites from BGC features; trained on 314 fungal BGCs they reached balanced accuracies of 51–68%, and adding 1,003 bacterial BGCs moved this only to 56–68%. That second result is our point of departure: if the bottleneck were a shortage of fungal examples, a fourfold increase in (cross-domain) training data should help substantially. It did not. (The two ceilings measure different quantities—Riedling et al. (2024) classify bioactivity, we reconstruct structure—but we argue both reflect the same grammar mismatch, and we draw no direct numerical comparison.)

The deeper issue is the framing. *Natural-product discovery should not be posed as direct* BGC$\rightarrow$ *structure regression, but as verifier-grounded inference over—and design upon—latent biosynthetic programs constrained by genome-derived grammars and physical observables.* A BGC is not a molecular blueprint but a constrained program space: the product is the image $y = \mathrm{Exec}_\Theta(z; G)$ of a latent program $z$ under a biochemical operator library $\Theta$ and a domain-derived grammar $G$. We realize this as a **grammar-agnostic biosynthetic inverse compiler** (Figure 1) and run it in three directions on one spine: a grammar-specific executor enumerates legal programs *forward*; a shared observability layer and verifier collapse them against mass and MS/MS *inverse*, returning a three-state verdict; and run *backward*, the same grammar places a desired molecule on a realizability geometry against the natural manifold and ranks the experiments that would confirm it (Section 9). In this sense the framework is *neuro-symbolic*: symbolic execution and verification enforce mass balance, legal reaction structure, and observable consistency, while learned models are reserved for the genuinely high-dimensional pieces—sequence-dependent activity, stereochemistry, and hidden transition policies. More than a predictor, the executor is then a *substrate on which other models compose soundly*: a learned sequence policy can rerank within the legal set $Z^\star$, a bioactivity model can filter it, and wet-lab build outcomes can update the realization cost—each inheriting the executor's soundness rather than eroding it. This is the middle the field lacks: generative chemistry yields molecules with no construction plan or feasibility guarantee, retrosynthesis searches reactions to commercial reagents but ignores enzyme space, and genome mining asks only what nature already makes. An executable, cost-weighted definition of biosynthetically realizable space—each point carrying a construction program $z$ and a verifying observable—is what a *target-directed* search over makeable chemistry requires. This stance answers a field diagnosis rather than ignoring it: Cox's review of iterative highly-reducing PKS programming finds it an emergent property and rational engineering "extremely difficult," naming machine learning as the way forward (Cox, 2023)—which we read as a design *constraint*, not an endorsement, requiring the architecture to stay *sound* precisely where rational reprogramming is intractable. We also scope the contribution honestly: structure elucidation in this field proceeds through heterologous expression and NMR with mass spectrometry as triage, so we build a metabologenomics *triage* tool grounded in those observables that names explicitly the clusters where triage is insufficient and structure elucidation is required. We do *not* claim a complete genome-to-structure predictor or a replacement for structure elucidation; we contribute the architecture, a characterization of the natural manifold that explains the field's

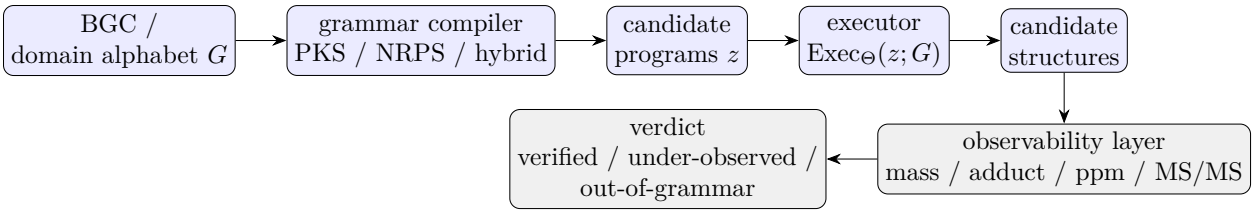


Figure 1: The grammar-agnostic inverse compiler. Family-specific grammars (blue) compile a cluster's domain alphabet into candidate programs that a sound executor renders to structures; a shared observability layer (grey) collapses the candidate set with mass and MS/MS, and a shared verifier returns a three-state reconstructibility verdict. Only the grammar and executor are family-specific; the observability and verdict layers are reused across PKS, NRPS, and hybrid grammars.

ceiling, a realizability geometry that turns the same engine into a design tool, and cross-grammar demonstrations. Our contributions are:

1. A **grammar-agnostic inverse compiler** over biosynthetic program grammars: a sound deterministic executor and beam verifier returning the exact consistent program set $Z^\star$ behind one observable-constrained interface (Sections 3–4).
2. A **deterministic reconstructibility analysis** of fungal iterative PKS over MIBiG 4.0 (Section 6): exact-match 8/326, a core-match *sensitivity* showing structural-similarity proxies are unreliable, and a tailoring-latent reformulation (7/106; 64% conditional on a producible core), with a curated literature benchmark (Section 5) and a cross-taxa transfer probe of the iteration-grammar wall (Section 6).
3. A **shared metabolomics observability layer** (Section 7): an adduct-aware, ppm-tolerant approximate verifier $Z^\star_\epsilon$, a within-grammar formula-ambiguity report, a tolerance-scored MS/MS matcher, and a minimum-sufficient-observables planner.
4. **Cross-grammar demonstrations** on PKS, NRPS, and a typed PKS–NRPS hybrid bridge (Section 8)—the same observability and verdict layers, only the program generator changing.
5. A **realizability geometry for biosynthetic design** (Section 9): every target molecule scored by its edit distance to the natural manifold on two typed axes—*structural* (domain content, with bacterial modular-PKS precedent) and *control* (the iteration program, the 0.567 frontier)—under a *literature-curated, super-additive* cost that encodes the field's measured engineering difficulty, with a worked design neighbourhood, the control-axis dominance result (69 : 49), and an **experiment-selection planner** that ranks the next observation by expected collapse of $Z^\star$ per unit cost and names the elucidation-required designs.
6. An **identifiability–controllability duality** (Theorem 1, §10): the inference resolution $Z^\star$ and the design resolution $D(y; O)$ are the same equivalence class $[z]_O$, so prediction ambiguity and design specification freedom are one quantity $\kappa_O$ at one observability frontier $\mathcal{F}_O$; instantiated on PKS, NRPS, and hybrid grammars and predicted to survive on RiPP (Corollary 3).
7. A **three-state reconstructibility verdict** (VERIFIED / UNDER-OBSERVED / OUT-OF-GRAMMAR) in which the UNDER-OBSERVED state names the next observable that would collapse $Z^\star$, resting on a failure-mode taxonomy (operator coverage, hidden iterative control, tailoring under-constraint, non-local structure) that replaces a single accuracy score.

All numbers in this paper are produced by the open code base by a single documented command and are traceable to a git commit; Appendix A lists the commands.

## 2 Background and the grammar-mismatch hypothesis

**Modular vs. iterative assembly lines.** Bacterial type I modular PKS—the archetype being the 6-deoxyerythronolide B synthase—are *colinear*. Each elongation module carries its own ketosynthase (KS) and acyltransferase (AT) for chain extension and an optional reductive cassette of ketoreductase (KR), dehydratase (DH) and enoylreductase (ER). The final product can be read off the linear order of modules. This clean structure-to-function map is why rule-based detectors such as antiSMASH (Blin et al., 2023) and PRISM (Skinnider et al., 2020) and the design database ClusterCAD (Eng et al., 2018; Tao et al., 2023) perform well on bacteria.

Fungal PKS are overwhelmingly *iterative*: a single set of catalytic domains is reused across $N$ cycles, and which reductive steps fire on which cycle is governed by a program that no current tool decodes. The highly-reducing (HR), non-reducing (NR) and partially-reducing (PR) subclasses behave like distinct programming languages. The lovastatin nonaketide synthase LovB runs eight cycles with a different reduction state each time, and its own ER domain is degenerate—ER activity is supplied *in trans* by the standalone partner LovC (Ma et al., 2009). In NR-PKS a dedicated product-template (PT) domain dictates the regioselectivity of aromatic-ring cyclization (Crawford et al., 2009).

**The hypothesis, sharpened to reconstructibility.** We hypothesize that the 51–68% ceiling reflects a control-flow mismatch rather than a sample-size limit. We state this as a *reconstructibility* (identifiability) property, not undecidability: throughout, "computable" is shorthand for whether the per-step core is recoverable from the BGC-visible domain content under our sound executor grammar. In colinear assembly lines the per-step core *is a function of domain content* and can therefore be tabulated (ClusterCAD); in iterative assembly lines the same domain set must perform different chemistry on different cycles, so the per-step core is *not* a function of domain content and the end-product-to-program map is many-to-one and unobserved. A model trained on colinear data learns a control flow that is not merely uninformative but incorrect for fungi—the near-flat cross-domain result of Riedling et al. (2024) is exactly the prediction this hypothesis makes. The rest of this paper turns the hypothesis into measurements.

## 3 A verifier-grounded executor

We represent the product of an iterative synthase as the output of a discrete latent *program* executed by a sound, deterministic chemical executor (Figure 2). A program is an ordered list of catalytic cycles,

$$z = \{(c_t, a_t)\}_{t=1}^{N}, \tag{1}$$

where $c_t$ marks a chain-elongation event and $a_t$ is an action over the reductive cascade and tailoring chemistry. The cascade is strictly ordered—ER presupposes DH presupposes KR—giving the four reachable $\beta$-states (keto, hydroxyl, enoyl, methylene) plus an optional $\alpha$-methylation (C-MeT) and a terminal chain release. The executor renders a program into a molecule, $\hat{y} = \mathrm{exec}(z)$.

**Deterministic core, not weights.** Each operator is a fixed reaction encoded as a reaction SMARTS and applied with RDKit (Landrum et al., 2025): a decarboxylative Claisen condensation

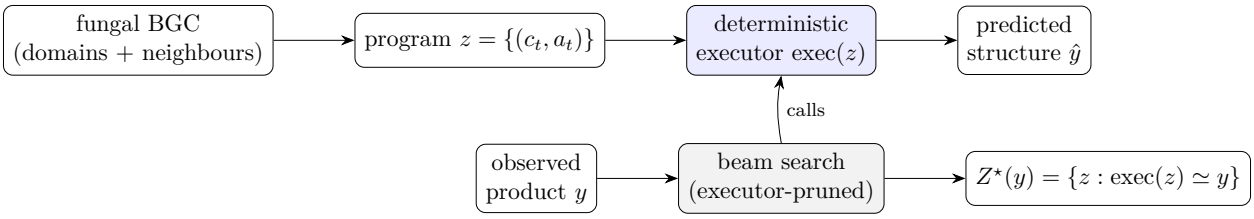


Figure 2: The executor (top) renders a program to a structure; the verifier (bottom) inverts it by beam-searching programs, executing each candidate and pruning any whose partial product cannot reach $y$, returning the exact consistent set $Z^\star$.

appends a $C_2$ ketide unit; KR reduces the $\beta$-ketothioester to the $\beta$-hydroxy; DH eliminates to the enoyl; ER reduces to the methylene; C-MeT $\alpha$-methylates; the thioesterase/cyclase releases the chain. The growing chain is modelled as an ACP-tethered thioester whose carrier is a single sentinel atom, so the unique active thioester anchors every operator unambiguously. The core is *validated*, not trained: we confirm each operator by exact mass bookkeeping (extension $+C_2H_2O$, KR $+H_2$, DH $-H_2O$, ER $+H_2$, C-MeT $+CH_2$) and by reconstructing known polyketides. Single-mode cyclization—lactonization, the small-aromatic aldol (orsellinic, 6-MSA) and the mellein dihydroisocoumarin—is implemented; the five-class PT-domain regiochemistry of large NR-PKS and the lovastatin Diels–Alder are deliberately scoped out and flagged. The executor is a pure, deterministic function validated within the project's regression suite (116 tests).

# 4 The verifier and identifiability

Because the executor is sound and deterministic and the program space is small ($N$ typically 4–10, roughly eight admissible actions per cycle), we recover programs by enumeration rather than amortized inference. Given a target $y$ and an operator set, a beam search emits programs, executes each (partial) program, and prunes any whose partial product cannot reach $y$. We accept a program into the *verified consistent set*

$$Z^\star = \{\, z : \text{exec}(z) \simeq y \,\}, \tag{2}$$

where $\simeq$ is graph isomorphism up to canonicalization. Two prunes make the search cheap: a carbon-count bound and an *analytic molecular-formula* prefilter that computes a program's released-acid formula from validated per-cycle deltas and only executes formula-consistent candidates (e.g. this takes the mellein search from 1,792 to 93 executor calls).

**Validation and identifiability.** On the curated benchmark the known program is contained in $Z^\star$ for all systems, and the search is deterministic and fast (mean 0.28 s/target). Observed degeneracy on the curated set is $|Z^\star| = 1$: programs are identifiable at these chain lengths. Genuine degeneracy *is* detectable when the operator set permits an alternative route—e.g. hexanoic acid is reached as acetyl + 2×ER or butyryl + 1×ER, giving $|Z^\star| = 2$—so identifiability is governed by the operator-set constraints, as the formulation predicts.

## 4.1 Guarantees

All guarantees are stated relative to the operator library $\Theta$, the (domain-derived) grammar $G$, the observable set $O$, and—for the tolerant mass observable (Section 7)—the mass tolerance $\epsilon$ and the adduct model. For $Z^\star_\Theta(y; G, O) = \{\, z \in Z_\Theta(G, O) : \text{Exec}_\Theta(z; G) \simeq y \,\}$ the search is *sound*—every

Table 1: Curated benchmark (excerpt). Reduction states: K keto, H hydroxyl, E enoyl, M methylene. Gate-tier entries round-trip exactly through the executor.

| System | Subclass | Program (starter + cycles) | Tier |
|---|---|---|---|
| Triacetic acid lactone | NR | acetyl + K,K → lactone | gate (sanity) |
| Orsellinic acid | NR | acetyl + K,K,K → aldol | gate |
| 6-Methylsalicylic acid | PR | acetyl + K,H,K → aldol | gate |
| Mellein | PR | acetyl + H,K,H,K → dihydroisocoumarin | gate |
| 6-Hydroxymellein | PR | acetyl + H,K,K,K → dihydroisocoumarin | gate |
| 2-Methylbutyric acid (LovF) | HR | acetyl + $M_{+Me}$ | gate |
| LovB nonaketide | HR | acetyl + 8 cycles (+DA) | hard / Phase-2 |
| Tenellin backbone (TenS) | HR | acetyl + 4 cycles (+NRPS) | Phase-2 trace |

returned $z$ executes to a structure matching $y$ under $\simeq$—and *relatively complete*: every legal program in the enumerated grammar that matches $y$ is returned, up to the beam bound. It is deliberately *not* biologically complete: if nature uses an operator outside $\Theta$, an empty $Z^\star$ is correct relative to $\Theta$ even though the molecule is producible *in vivo*. An empty consistent set is therefore a typed *certificate of failure mode*—missing operator, hidden control, tailoring under-constraint, or non-local pathway—not a claim of biological impossibility; the approximate verifier $Z^\star_\epsilon$ inherits the same relativity, now also to $\epsilon$ and the allowed adducts. Because the operator library is modular, extending the engine to complex polycyclic cascades, trans-acting tailoring events, or additional release chemistries does not require changing the shared verifier or observability layers; it requires registering new typed operators in the relevant family-specific executor.

## 5 A curated benchmark of fungal PKS programs

We hand-curate twelve literature-sourced fungal polyketide programs (Table 1), each a machine-readable specification of starter, per-cycle reduction/methylation, and release, with a literature citation, a provenance tag (*independent* ground truth vs. *pending* an authoritative structure), and a confidence flag. Six gate-tier entries with independent ground truth round-trip exactly through the executor; sanity entries cover the reductive extremes; harder systems (the LovB nonaketide; the TenS PKS–NRPS backbone) are retained as trace-level assets.

A finding from curation is worth stating: the executor caught a mass-balance inconsistency in a literature-pass program for mellein. The single-ketoreduction program reconstructs 6-hydroxymellein ($C_{10}H_{10}O_4$), not mellein ($C_{10}H_{10}O_3$); mellein requires a *second* ketoreduction. The verifier earns its keep by making such errors impossible to overlook.

## 6 Characterizing the natural manifold

The natural fungal PKS manifold has **eight exactly-reachable cores**, a **0.567 control-axis ceiling with a structural mechanistic explanation**, and a **tailoring-permissive interior with 64% conditional reconstructibility** — this section characterizes its shape. We read these results not as a catalogue of where prediction fails but as the geometry the design contribution of §9 operates over: the boundary of what nature builds, the axis along which it is hard to read, and the interior where it is tractable once a core is in hand.

### 6.1 The boundary: eight exactly-reachable cores

The natural manifold has **eight exactly-reachable cores**, defining its boundary: of the 326 fungal PKS entries in MIBiG 4.0, exactly eight are placed exactly by the sound grammar run inverse. This count is a lower bound — the missing operators are enumerable (PT multi-ring aromatic closures, oxidative rearrangements), so coverage moves it up — and it is a *boundary condition*, not an accuracy: the set of cores the current sound grammar places exactly, the edge from which §9 measures design distance. "Only 8" is the wrong frame; "the manifold's boundary is here, and it is sharp" is the right one.

### 6.2 Similarity is not a reconstructibility metric

A natural temptation is to soften the boundary with a structural-similarity proxy — count a core "reconstructed" if a predicted skeleton embeds in the product. We show this proxy is *unreliable*: it swings from 8 to 160 reconstructed cores as a single nuisance parameter (the allowed ring-closure slack) is varied, and it admits non-PKS false positives (a de-heteroatomed alkaloid skeleton can embed in a polyketide chain). Similarity is therefore not a reconstructibility metric; we keep exact match as the airtight signal and use the tailoring-latent reformulation below as the principled relaxation, not a tunable proxy.

### 6.3 The tailoring-permissive interior

Most MIBiG products are a PKS core plus post-PKS tailoring, so exact core-match understates reconstructibility. Moving tailoring into the latent — $z = (z_{\text{PKS}}, z_{\text{tail}})$ with a gene- and formula-budgeted edit applier — the joint search exactly reconstructs **7/106** annotated clusters overall, but the decisive number is conditional: of the 99 failures, **95 are PKS-core-unreachable** (no producible backbone), and among the clusters where a core *is* producible the tailoring search succeeds near-uniquely at **64%**. The bottleneck is PKS-core cyclization coverage, not tailoring. So the manifold's interior is *tailoring-permissive*: once a core is in reach, the decorations are near-uniquely reconstructible, and the residual hardness is concentrated on the core, not the periphery.

### 6.4 The hard axis: the iteration-grammar wall, structurally explained

The manifold's one genuinely hard axis is per-cycle control. A cross-taxa transfer probe makes it quantitative: the per-step reduction chemistry is learnable from abundant bacterial modular data and transfers to fungal cycles *perfectly* when each cycle's active reductive domains are given (1.00), but collapses to **0.567** when only the synthase's fixed domain set is given — the iteration-grammar wall. The wall decomposes into a narrow highly-reducing per-cycle-reduction control wall (∼9% of products) and a broad, coverage-addressable aromatic-cyclization wall (∼61%).

**Why the wall is structural, not a data gap.** A fungal iterative synthase reuses one set of catalytic domains across every cycle; the per-cycle outcome is fixed by the chain-length and substrate context the enzyme sees on that pass, not by any per-cycle change in the (constant) machinery. This is the precise form of Cox's observation that highly-reducing PKS programming is an emergent property that "will remain extremely difficult" to predict rationally (Cox, 2023): it is emergent from the interaction between the constant catalytic machinery and the changing substrate context across cycles. The 0.567 is therefore not a placeholder for missing work; it is a structural ceiling for any predictor whose input is the synthase's fixed domain content. A positive/negative control on the same predictor isolates this. Over 141 colinear bacterial clusters, predicting each module's

reduction from its *own* per-module domains reaches **0.78**; predicting it from the synthase's single *pooled* domain set—the condition a fungal iterative synthase is in by construction—collapses to **0.50** (the modal baseline), and the fungal 0.567 sits in that pooled column. One predictor nearly doubles when given a per-step signal and falls to the wall when denied it: the deficit is the missing per-step *input*, not the model. (Whole-program exact recovery is only 0.17 even on bacteria, as per-module errors compound across the assembly line—which is why a per-step *sequence* signal helps precisely where one exists, and cannot where it does not.)

**A sequence-level probe separates two questions.** The cross-taxa probe folded together two questions the sequence head now lets us separate: *(i)* does protein *sequence* carry the per-step chemistry signal the domain abstraction discards, and *(ii)* would that signal, even if real, close the *fungal* wall? *(i) Yes, empirically*, on bacterial *modular* PKS, where each module carries its own ketoreductase (KR) sequence: under homology-clean evaluation at 70% sequence identity,[1] a frozen ESM-2 head extracts inactive-domain and stereochemistry signal at balanced accuracy $\sim$**0.85** (AUC 0.91 for inactive-domain detection), decisively above the chance baselines of 0.50–0.53 the domain cartoon is pinned to. *(ii) No, structurally*: a fungal iterative synthase reuses *one* KR sequence across all its cycles, so a per-domain sequence head emits a single prediction per synthase—exactly the fixed-domain condition that yields 0.567. Sequence carries per-step chemistry where there *is* a per-step sequence (bacterial, modular) and cannot where there is not (fungal, iterative); the wall is structural, not a data gap.

This manifold — bounded by eight reachable cores, structured by the iteration-grammar wall, with a defined tailoring-permissive interior — is the object the design contribution of §9 operates over. Its hard axis is exactly the axis along which the interesting designs live (§9.6), which is why the same measurement reads as a prediction ceiling here and a design frontier there.

## 7 Mining: genome and metabolome

The same sound executor, run *forward* as a generator rather than inverted against a known product, is an operational genome-mining tool. Because the executor is a *sound generator*, a cluster's catalytic alphabet bounds a small enumerable set of producible cores—tiny when reduction is architecturally fixed, combinatorial for highly-reducing programs (a $\sim 13\times$ candidate-set ratio in our curated test)—which can be ranked and matched against metabolomics, recasting the reconstructibility ceiling as a constrained hypothesis generator. Genome mining is then inference over the latent program $z$ from whatever observables are in hand, and the residual candidate set—which we measure operationally by its size, not by a calibrated entropy—falls as orthogonal observables are added (Table 2).

**The observable ladder.** The genome alone leaves the program highly uncertain (the 0.567 per-cycle transfer); the domain alphabet bounds the legal operators; the MS-measured molecular formula pins the reduction *count*. Conditioning the generator on the two observables a metabologenomics pipeline actually supplies—domain alphabet and mass—collapses our curated reachable set to a median of *one* candidate core (top-3 recall 9/10), so the per-cycle reduction combinatorics,

[1] Leave-cluster-out by sequence-identity cluster; identity computed via BLOSUM62 global alignment in Biopython, verified at self-identity 1.00 and an observed cross-cluster identity floor of 0.35. A strict 30%-identity partition is undefined here: KR is a single conserved Rossmann-fold family with a $\sim$35% pairwise-identity floor, so it cannot be split into remote-homolog folds — the head's generalization across remote KR homologs is therefore not assessable on this dataset, a statement about the protein family rather than the method.

Table 2: The engine's input/output contract: each observable and what it constrains. Rows distinguish retrospective verification (known product) from prospective mining (unknown product).

| Observable | Constrains | If absent / noisy |
|---|---|---|
| domain alphabet | legal operators (grammar) | broad candidate set |
| domain order / subclass | macro-grammar, release mode | wrong grammar |
| accurate mass (formula) | reduction / edit budget | formula-isomer ambiguity |
| MS/MS fragments | skeleton / isomer identity | residual isomers unresolved |
| co-clustered tailoring genes | admissible edit families | over/under-constrained tailoring |
| known product (benchmark only) | retrospective verification | prospective mining mode |

and with them the 0.567 grammar wall, largely dissolve once alphabet and mass are combined. Mass alone is not always sufficient: for small cores within executor reach alphabet+mass is near-deterministic, but larger cores under a rich grammar retain formula-isomers, and these are *fragmentation*-distinguishable (a crude single-bond-cleavage fingerprint already gives all 112 formula-isomers of one $C_{12}$ core distinct fragment sets), so predicted MS/MS resolves the residual to a single core—a domain→mass→fragmentation pipeline grounded throughout in the sound verifier. The binding constraint is then cyclization *coverage*, not computability.

**A grammar-relative identifiability benchmark.** A controlled benchmark over 30 sampled cores bears the ladder out: the median verified set collapses $1672 \to 219 \to 1$ across grammar → +mass → +MS/MS, and the true core is recovered 30/30 under the correct grammar and true mass but 0/30 under a deliberately wrong, non-reducing grammar—so the grammar and the mass each do real work, and the small sets are not an artefact of a tiny search space. The intermediate 219 is the complex-core regime in which mass narrows sharply but not uniquely and MS/MS supplies the uniqueness; the median-1 collapse above is the tight-alphabet small-aromatic regime. We release a reference implementation, `lpi.engine`: a single interface mapping a cluster's domain set and observables (accurate mass, MS/MS) to ranked candidate cores and a per-cluster reconstructibility verdict.

**A first real-cluster case study.** To test the engine against the outside world rather than only curated or simulated targets, we ran it *blind* on a real MIBiG cluster—the 6-methylsalicylic acid synthase (`BGC0001275`, producer *Glarea lozoyensis*). MIBiG annotates the cluster only as class "PKS" with no per-module domains—the very gap this paper documents—so the grammar input is the canonical 6-MSAS architecture (KS-AT-DH-KR-ACP), and the sole observable is the real monoisotopic mass (152.0473, PubChem CID 11279) supplied as the $[M-H]^-$ ion at 5 ppm; the product structure is withheld from inference. The domain alphabet alone admits 1,121 producible cores, the accurate mass collapses these to a single candidate, and the engine returns VERIFIED. Only on reveal do we score it: the unique candidate is exactly 6-methylsalicylic acid (rank 1). A real EI-MS spectrum (MassBank `MSBNK-Fac_Eng_Univ_Tokyo-JP008039`) corroborates the molecular ion at $m/z$ 152; being EI rather than LC-MS/MS it serves as a mass sanity reference, not a fragmentation rung, and since the mass alone is already decisive we report the MS/MS rung as not applied rather than simulate it. This is one clean real-cluster case, not a benchmark.

# 8 Generality across grammars

This section is an architectural proof of concept: the NRPS and hybrid candidate sets below are small *by construction* (a minimal monomer set), and the claim is generality of the shared inference-and-verdict *interface*, not biochemical coverage. The program-synthesis view of Section 1—a BGC is not a molecular blueprint but a constrained program space, $y = \mathrm{Exec}_\Theta(z; G)$—predicts that the engine should not be specific to polyketides: biosynthetic families differ in their program grammars and operator libraries $\Theta$, but the *observability* layer (accurate mass, MS/MS) and the reconstructibility verdict are physical, not family-specific. We test this directly. Behind the same `infer_cluster`$(G, O) \to (Z^\star$, state, next observable) interface we implement a minimal non-ribosomal peptide (NRPS) grammar: a pure, deterministic peptide executor (amide condensation of a small monomer set; linear and head-to-tail macrocyclic release) with an exact residue-formula prefilter, presented through the identical `Grammar` protocol as the PKS generator (Table 3).

Table 3: One inference interface, two grammars. Verified candidate-set size after each observable rung, and the three-state verdict; the same mass / MS/MS / verdict machinery serves both families.

| Cluster / product | Grammar | alphabet | +mass | +MS/MS | Verdict |
|---|---|---|---|---|---|
| orsellinic acid | PKS | 2 | 1 | 1 | verified |
| olivetolic acid | PKS | 5012 | 87 | 1 | verified |
| cyclo(Gly–Gly) | NRPS | 100 | 1 | 1 | verified |
| glycylglycine | NRPS | 576 | 1 | 1 | verified |
| cyclo(Phe–Tyr) | NRPS | 100 | 1 | 1 | verified |
| NRPS, mass withheld | NRPS | 788 | — | — | under-observed |

Because the observability layer consumes only executed candidate structures, the same ppm-tolerant mass recovery, adduct model, MS/MS scorer, observable ladder, and three-state verdict operate unchanged across PKS and NRPS grammars; only the program generator is family-specific. The withheld-mass row is the engine in its operative mode: from the genome alone the peptide is *under-observed*, and the verdict names the observable—accurate mass—that would collapse the candidate set.

**Scope.** This is a proof of *framework* generality, not NRPS biochemical coverage (the v0 grammar's conservative scope is stated once in Section 13): the point is that a second grammar plugs into the same inference and verdict stack, the architecture being grammar-agnostic because the shared layer reads only candidate structures.

## 8.1 Compositional grammars and hybrid biosynthesis

Biosynthetic families also *compose*. A PKS–NRPS hybrid is modelled as a typed composition of the two grammars,

$$z_{\text{hybrid}} = z_{\text{PKS}} \circ h_{\text{handoff}} \circ z_{\text{NRPS}} \circ r_{\text{release}}, \tag{3}$$

where $h_{\text{handoff}}$ is an explicit amide-condensation operator joining the polyketide *linear-acid checkpoint* (Section 3) to the peptide N-terminus. The hybrid grammar does not re-implement either family: it composes the PKS generator with the NRPS monomer set, and the same adduct/ppm-tolerant mass recovery, MS/MS scorer, observable ladder, and three-state verdict run over the composed candidates unchanged (a minimal hybrid, acetoacetyl-glycine, collapses genome→mass to a single *verified* core through the identical stack). Crucially, the fungal tetramic-acid Dieckmann release—the

step that converts the TenS polyketide–tyrosine intermediate into pretenellin A—is declared as a *typed but unimplemented* operator: requesting it returns an explicit out-of-grammar verdict naming the missing release operator, rather than a silent failure or a wrong structure. The scope boundary the rest of this paper describes in prose is thus made machine-checkable. The framework now spans **PKS, NRPS, and a minimal PKS–NRPS hybrid bridge** behind one inference-and-verdict interface; grammars are family-specific and composable, while observability and the reconstructibility verdict are shared.

# 9 Sound biosynthetic design over the iteration grammar

Running the executor forward turns a program into a structure; running the verifier inverse turns a structure into the set $Z^\star$ of programs that could have produced it. Here we run the grammar *backward*: given a desired, possibly non-natural target, we place it on a *realizability geometry* relative to the natural manifold, and we rank the experiments that would confirm it. Two results make this more than a restatement of the diagnostic half. First, the design space decomposes onto two axes that map to fundamentally different wet-lab capabilities. Second—and this is what keeps the contribution honest—the cost on those axes is the field's *measured* difficulty of fungal PKS engineering, curated from primary literature, not an assumption we supply.

## 9.1 The realizability geometry: two axes

A design target is a program $z_d$; its product is $y_d = \mathrm{Exec}_\Theta(z_d; G)$. We score $z_d$ by its edit distance to the nearest natural cluster on the manifold, decomposed into two typed axes:

- **Structural** edits change the *domain content*: add a reductive domain (`add_reductive`), swap the starter or extender (`starter`, `extender`), reprogram the release cyclase (`release`), or insert a methyltransferase de novo (`add_cmt`). These map onto bacterial modular-PKS engineering.
- **Control** edits change the *iteration program* with the domain set fixed: which cycle a present domain fires on (`reduction`, `c_methyl`) or the iteration count (`cycle_add`, `cycle_remove`). This is the iteration-grammar frontier—precisely the $1.00 \to 0.567$ wall of the cross-taxa transfer probe (§6), now read as a design dimension rather than a prediction limit.

A design at coordinate $(2, 1)$ (two structural, one control edit) is a different proposition from one at $(3, 0)$: the first requires reprogramming the iteration itself, the second is documented domain engineering. Each target's most-realizable route is the minimum-cost edit path to a natural cluster, and its *verdict*—NATURAL, ENGINEERABLE, FRONTIER, or SPECULATIVE—is read off that path. Crucially, the control axis *only exists as a design dimension* for a machine that distinguishes per-cycle firing within a fixed domain set; a tool without the iteration grammar cannot represent the edit "reprogram the ketoreductase at cycle 3," let alone cost it (§9.6).

## 9.2 A literature-grounded, super-additive cost

The design enumeration bounds the cost model to the nine edit kinds that actually appear in the 1-edit neighbourhood, and we curated each from primary sources rather than scraping abstracts (two tier rulings turned on facts a search snippet stated incorrectly). The structural edits inherit the mature combinatorial-biosynthesis canon: acyltransferase/extender swaps and loading-module/starter swaps are documented (Musiol-Kroll & Wohlleben, 2018; Jacobsen et al., 1997), as are reductive-loop transplants (Kellenberger et al., 2008); de novo C-methyltransferase insertion has no precedent (the methylation literature reprograms a *present* C-MeT, never inserts one) and is scored SPECULATIVE.

The control edits are the unsolved frontier. Cox's review of iterative highly-reducing PKS programming (Cox, 2023) states plainly that rational efforts "will remain extremely difficult" because the programme is "an emergent property… highly unpredictable." Yet a *single* edit beside a closely related cluster is achievable: domain swaps between the tenellin and desmethylbassianin synthases mapped methylation-pattern and chain-length changes cleanly (Fisch et al., 2011). We encode exactly this shape. Structural edits sum linearly by tier; control edits are **super-additive**, growing quadratically in the number $k_{\text{ctrl}}$ of control edits in a design:

$$\text{cost}(z_d) \;=\; \underbrace{\textstyle\sum_{e\in\text{struct}} \tau(e)}_{\text{linear}} \;+\; \underbrace{\Big(\textstyle\sum_{e\in\text{ctrl}} \tau(e)\Big)\cdot k_{\text{ctrl}}}_{\sim k_{\text{ctrl}}^2,\ \text{super-additive}}, \tag{4}$$

where $\tau$ is the curated per-edit tier. The control term multiplies a sum *already* taken over the $k_{\text{ctrl}}$ control edits by $k_{\text{ctrl}}$ again, so it scales as $k_{\text{ctrl}}^2$: one tier-2 control edit costs 2, whereas two tier-2 control edits cost $(2{+}2)\cdot 2 = 8$, not 4. Equation (4) thus reduces to the linear tier cost at $k_{\text{ctrl}} = 1$ (the Fisch–Cox single-edit regime) and grows quadratically as control edits stack (the Cox emergent regime). The cost model is therefore not asserted: it is the published difficulty landscape of fungal PKS engineering, written into the objective. This is the difference between "we built a design engine" and "we built a design engine whose cost model is the measured difficulty of the field."

### 9.3 The worked neighbourhood of 6-MSA

Figure 3 walks one clean template—the 6-methylsalicylic acid synthase, the cluster with a verified blind real-mass match (§7)—end to end. Each row of its 1-edit neighbourhood is a build proposal carrying its (structural, control) coordinate, realizability verdict, predicted monoisotopic mass, and the observable that would confirm it. The neighbourhood separates cleanly into an **engineerable** block (single structural edits—starter swaps such as acetyl→hexanoyl at mass 244.13, release reprograms—all confirmable by accurate mass) and a **frontier** block (single control edits such as reprogramming the cycle-3 reduction), with coverage-limited out-of-grammar designs sunk below the headline. One row reads, "one documented edit (acetyl→hexanoyl) from the 6-MSA synthase, mass 244.13, verifiable by accurate mass"; another, "one iteration-program edit (reprogram cycle-3 reduction), mass 172.07, needs MS/MS." Design distance, the predicted observable, and a verdict on whether triage suffices or structure elucidation is required—on the same line.

### 9.4 Control-axis dominance

Across the 1-edit neighbourhood of the natural manifold the control axis dominates: **69** designs lie on the frontier (involve a control edit) against **49** engineerable (structural-only), with 24 speculative. That ratio is itself a product of the literature curation—the raw, uncurated count was a starker 93 : 34, and grounding the edit costs (demoting release to a structural domain swap, raising de novo C-MeT and iteration-count edits) softened it to a defensible margin. Fungal biosynthetic *design* is mostly iteration-program editing: the axis only a sound per-cycle executor can express.

### 9.5 The experiment-selection planner

Given a design, which experiment should one run next? The planner ranks each modelled observation $o$ by expected information gain per unit cost, weighted by realizability:

$$\text{score}(o) \;=\; \mathbb{E}[\Delta|Z^\star|]\cdot w_{\text{real}}/c_{\text{obs}}(o), \qquad w_{\text{real}} = 1/\text{cost}(\text{unresolved edits}), \tag{5}$$

| $(S, C)$ | 1-edit design from the 6-MSA synthase | mass (Da) | verified by |
|---|---|---|---|
| ENGINEERABLE — structural-only, documented domain engineering (9 designs) | | | |
| $(1, 0)$ | starter swap acetyl→hexanoyl | 244.13 | accurate mass |
| $(1, 0)$ | starter swap acetyl→butyryl | 216.10 | accurate mass |
| $(1, 0)$ | starter swap acetyl→propionyl | 202.08 | accurate mass |
| $(1, 0)$ | release reprogram (aromatic→lactone) | 188.07 | accurate mass |
| FRONTIER — $\geq 1$ control edit, iteration-program editing (12 designs) | | | |
| $(0, 1)$ | reprogram cycle-3 reduction (keto→DH) | 172.07 | + MS/MS |
| $(0, 1)$ | cycle-2 reduction KR→DH | 170.06 | accurate mass |
| $(0, 1)$ | cycle-1 reduction keto→KR | 190.08 | accurate mass |
| $(0, 1)$ | add a fourth cycle (+cyc4) | 230.08 | accurate mass |

Figure 3: The worked 6-MSA design neighbourhood (`scripts/worked_design_6msa.py`): each row is a build proposal carrying its (structural, control) edit coordinate, predicted monoisotopic mass, and the observable that would confirm it. The two blocks are different *kinds* of proposal at comparable cost: a $(1, 0)$ starter swap (acetyl→hexanoyl) is documented domain engineering a wet-lab builds today; a $(0, 1)$ cycle-3 reduction reprogram is the iteration-grammar frontier—the axis only a sound per-cycle executor can express. Of the full neighbourhood, 9 are engineerable, 12 frontier, and 4 speculative (de-novo C-MeT, shortened chain). Design distance and verification plan on one line.

with speculative designs hard-excluded ($w_{\text{real}} = 0$). The two cost terms sit on different sides: observation cost $c_{\text{obs}}$ (accurate mass 1, MS/MS 3, in committed relative units) in the denominator, and the super-additive realizability cost of Eq. (4) as the design-side weight. Because that weight is super-additive, resolving one of two stacked control edits drops the residual penalty non-linearly—so the planner reasons about the *joint resolution state* of a design, not merely the most-uncertain edge. The planner's output is a decision tree over observation outcomes in which both children of every node are first-class: the refuted branch (the observation is inconsistent with the predicted product) carries as much information as the confirmed one.

The planner returns a typed triage verdict. VERIFIED-BY-TRIAGE: the modelled observables collapse $|Z^\star|$ to near-unique. ELUCIDATION-REQUIRED: the modelled observables are exhausted and $|Z^\star|$ is still large—the design must go to structure elucidation. OUT-OF-GRAMMAR: a mass observation refutes every legal program. *Naming the elucidation-required case is the contribution*: a triage tool that knows its own limits, and whose limits are typed and machine-checkable. Figure 4 contrasts two trees: 6-MSA collapses at accurate mass (a shallow tree—the planner doing its job), while the highly-reducing $C_{12}$ core BAB stays under-determined through mass and MS/MS ($4050 \to 25 \to 23$ candidates) and is routed to expression + NMR (the planner naming its limit).

This scoping is forced by the data, not chosen for modesty. All seven verified products in our corpus had their genome→structure link established by heterologous expression (with knockout or native expression) and NMR, with accurate mass and MS/MS serving as triage and detection—never as the sole structure-determining observable. The planner is therefore a metabologenomics *triage* tool: it answers "which observation collapses $|Z^\star|$ given the genome," not "how is this structure proven." Evaluated retrospectively against this curated corpus it routes correctly—the small aromatics (orsellinic acid, 6-MSA, mellein, 6-hydroxymellein) verify by accurate mass, while the highly-reducing core is flagged elucidation-required—and on the design neighbourhood it steers attention to engineerable and frontier designs over speculative ones.

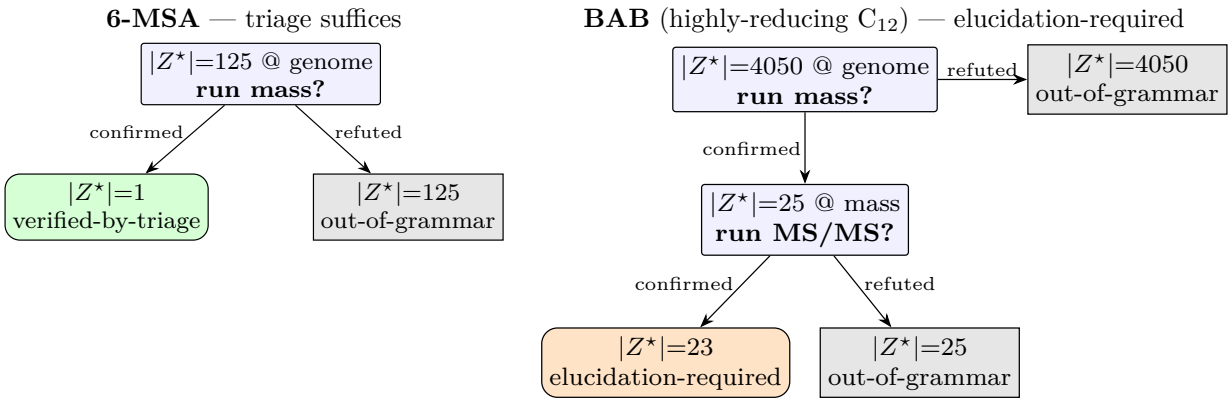


Figure 4: The triage planner doing its job (6-MSA, left: accurate mass collapses $|Z^\star|$ to a unique core $\Rightarrow$ VERIFIED-BY-TRIAGE) and naming its own limit (BAB, right: mass then MS/MS leave $|Z^\star|$ at 23 $\Rightarrow$ ELUCIDATION-REQUIRED, routed to expression+NMR). Refuted branches (OUT-OF-GRAMMAR) are rendered first-class at every node, not collapsed. Source: `scripts/planner_tree_figure.py`.

### 9.6 The architectural moat

The contribution is architectural, not a capability we happen to have: the control axis exists as a design dimension only for a sound executor that distinguishes per-cycle firing within a fixed domain set. "Reprogram the ketoreductase at cycle 3" is not a hard edit to a retrosynthesis tool (which works in chemical space), an enzyme-design tool (which works per protein), or a pathway tool (which ignores assembly-line grammar)—it is a sentence about an object none of them can represent. We are the missing layer between generative chemistry and protein design, and the cost on the axis is the field's measured difficulty (Eq. (4)). That the same iteration-grammar wall is a prediction ceiling read one way (§6) and a design frontier read the other is no coincidence but a duality, stated and proved in §10: prediction ambiguity and design freedom are one observability quantity $\kappa_O$.

## 10 Identifiability–controllability duality

Section 6 read the iteration-grammar wall as a ceiling on *inference*; §9 read the same wall as the *design* frontier. That one measurement carries two valences (§9.6) is not a rhetorical observation but a property of the program-grammar setup itself. Stated cleanly it is a duality between identifiability and controllability that survives generalisation beyond fungal PKS, and we use it to predict where the architecture should reproduce its results on families we have not implemented.

**Setup.** Fix a grammar $G$ with program space $\mathcal{Z}(G)$, an operator library $\Theta$, a sound executor $\mathrm{Exec}_\Theta(\cdot\,; G) : \mathcal{Z}(G) \to \mathcal{Y}$ into a structure space $\mathcal{Y}$, and an observable set $O$ with a projection $\mathrm{obs}_O : \mathcal{Y} \to \mathcal{S}$ onto a signature space $\mathcal{S}$ (mass, MS/MS fragmentation, domain alphabet). Composition gives the observable map of a program, $\phi_O = \mathrm{obs}_O \circ \mathrm{Exec}_\Theta : \mathcal{Z}(G) \to \mathcal{S}$. Two programs are *$O$-equivalent*, $z_1 \sim_O z_2$, iff $\phi_O(z_1) = \phi_O(z_2)$; the class $[z]_O \subseteq \mathcal{Z}(G)$ is the set of programs the chosen observables cannot tell apart from $z$. Two operational quantities use this directly. The *inference resolution* at a true program $z$ under $O$ is the verifier's consistent set of §4,

$$Z^\star(z; O) = \{\, z' \in \mathcal{Z}(G) : \phi_O(z') = \phi_O(z) \,\} = [z]_O,$$

and the *design resolution* at a target structure $y$ specified through $O$—the programs an engineer may pick from when only the observable signature of $y$ is fixed—is $D(y; O) = \phi_O^{-1}(\mathrm{obs}_O(y))$.

**Theorem 1** (identifiability–controllability duality)**.** For every program $z \in \mathcal{Z}(G)$ with structural image $y = \mathrm{Exec}_\Theta(z; G)$,

$$\boxed{Z^\star(z; O) \;=\; [z]_O \;=\; D(y; O).}$$

The verifier's consistent set when $y$ is the true product, the observable equivalence class of $z$, and the set of programs an observable specification of $y$ admits as build candidates are one set. Its cardinality $\kappa_O(z) = |[z]_O|$ is simultaneously the inference ambiguity floor at $z$ and the design specification freedom at $y$: no inference scheme can return fewer than $\kappa_O$ candidates under $O$, and no design specification limited to $O$ can rule among them without further observation.

*Proof.* The three sets are equal as preimages of $\phi_O(z)$ under $\phi_O$; the content is in the identifications. *(i)* $Z^\star(z; O)$ of §4 is the verifier's algorithmic output: by soundness of the executor (§3) every returned $z'$ satisfies $\mathrm{Exec}(z') \simeq y$, and by relative completeness (§4.1) every legal program with that property is returned, up to the (here saturating) beam bound. Together these state that $Z^\star(z; O)$ is exactly the preimage $[z]_O$ as a set, not merely up to notation. *(ii)* $D(y; O)$ is the set of programs an engineer working from an observable specification of $y$ can propose: any $z' \in [z]_O$ produces the same signature as $z$ and is therefore indistinguishable from it under the specification, while any $z' \notin [z]_O$ violates it, so $D(y; O) = [z]_O$ in the operational sense of "candidates $O$ alone cannot rule out." The verifier's algorithmic output and the designer's specification space are thus identical as sets; the non-triviality is the identification of two operationally distinct objects with one equivalence class. □

**Notation.** We write $\kappa_O(y) := \kappa_O(z)$ for any $z \in \mathrm{Exec}^{-1}(y)$; the theorem makes this well-defined, since two programs with the same structure share a signature and hence a class.

**Definition 1** (observability frontier)**.** For $(G, \Theta, O)$ the *observability frontier* is

$$\mathcal{F}_O(G, \Theta) = \{\, y \in \mathcal{Y} : \kappa_O(y) > 1 \,\}, \tag{6}$$

the structures the observables cannot uniquely program. Theorem 1 reads at the level of $\mathcal{F}_O$: it is at once the under-observed set for inference and the multiply-routable set for design, so the triage planner of §9.5 and a design-specification planner are one query on this object, parameterised by $O$.

**Corollary 1** (the planner is already a design planner)**.** The triage planner of §9.5, *unmodified*, ranks experiments for design specification and for inference identification at once. By the duality the ranking $\mathbb{E}[\Delta|Z^\star|]/c_{\mathrm{obs}}(o)$ of Eq. (5) is identical to the ranking of observations by expected reduction in *design* ambiguity per unit cost; the framework needs no design-side analogue, and the routing verdicts (VERIFIED-BY-TRIAGE / ELUCIDATION-REQUIRED) carry across to design with no machinery change.

**Corollary 2** (the iteration-grammar wall, both ways)**.** Under $O = \{\text{domain alphabet}\}$, externalised per-module state in bacterial modular PKS gives $\kappa_O \approx 1$ (the 1.00 cross-taxa transfer of §6.4): inference returns single programs, and a structural specification maps to a unique build route. Compressed, state-dependent control in fungal iterative PKS gives $\kappa_O > 1$ at the same observable (the 0.567 collapse): inference is under-observed, and design admits multiple compositionally degenerate routes—the *existence* of the control axis as a design dimension (§9.1) is the duality read backward, available only to a representation that preserves the equivalence class rather than collapsing it. The asymmetry between the modular and iterative engineering literatures (Yuzawa et al., 2018; Tao et al., 2023; Cox, 2023) is one quantity at one observable, read in opposite directions.

**Corollary 3** (cross-family prediction)**.** The proof uses only the executor's soundness and the verifier's relative completeness, stated for an abstract $(G, \Theta, O)$ in §4.1, so the duality transfers to any family the framework can express. In ribosomal peptide (RiPP) biosynthesis the precursor peptide is genome-readable (small $\kappa_O$ on core composition) while post-translational modification order is iteration-grammar–like (large $\kappa_O$ on modification topology); the duality predicts core-residue substitutions are *both* highly identifiable and straightforward to engineer, while modification-topology edits are *both* prediction-limited and design-limited at the same frontier $\mathcal{F}_O$. This is the pattern in the lanthipeptide (Repka et al., 2017) and cyanobactin (Donia et al., 2008) engineering literatures—routine core-residue swaps, a barrier at modification-pattern reprogramming—and the duality's content is that the two barriers are not coincidental but one quantity $\kappa_O$ on one frontier. It is falsifiable on the NRPS/RiPP chassis of §8: instantiate a RiPP grammar and compute $\kappa_O$ before and after modification observables.

**Remark.** The duality reframes the field's central diagnostic: fungal biosynthesis is not *separately* hard to predict and hard to engineer, but observability-limited, the two hardnesses being projections of one quantity onto two questions. The v1 programme follows—characterise $\mathcal{F}_O(G, \Theta)$ as a function of $O$ for each grammar; the shared layer of §7 implements the query for $O \in \{\text{mass}, \text{MS/MS}\}$, and widening $O$ until $\mathcal{F}_O$ shrinks to where one wishes to operate is then a finite programme on a defined object. We instantiate the duality on the three grammars implemented here (PKS, NRPS, hybrid) and predict its survival on RiPP via Corollary 3; we claim no theorem over an abstract category of program grammars, only that the proof covers any $(G, \Theta, O)$ meeting the soundness and relative-completeness conditions of §4.1.

## 11 Empirical structural rescue: a dual-engine resolution paradigm

The Outlook (§15) predicts that the fungal per-cycle policy is $a_t \sim P(a_t \mid G, s_t)$: the static enzyme $G$ and an evolving substrate state $s_t$ jointly determine the action. We take a first empirical step toward that injection, and the result is sharper than a single "joint necessity" claim: the same loss decomposes into *two complementary resolution engines*, each precisely applicable to a different chemical regime of the candidate set.

**The corpus.** We extend the curated 30-cycle HR/PR set (§5) by mining MIBiG fungal PKS clusters whose engine-derived $|Z^\star|$ at the metabologenomics observable set (alphabet + molecular formula) lands in the constrained mode, $|Z^\star| \leq 10$. An RDKit-driven SMILES-to-formula recovery pass rescues the 112 MIBiG entries without a registered formula; the tractable inventory fixes at 74 clusters and the $|Z^\star|$ histogram is bi-modal with an empty middle at $|Z^\star| \in [11, 24]$ across all 74—a structural cliff in the natural manifold (§6) that survives the recovery and is not a data-hygiene artifact. Of the 74, 7 land at $|Z^\star| = 1$ (joining the curated 30) and 9 at $2 \leq |Z^\star| \leq 10$ contribute the silver tier. A verifier-constrained marginal- likelihood loss $\mathcal{L}(\theta) = -\sum_B \log \sum_{z \in Z^\star(B)} \pi_\theta(z \mid B)$ collapses to standard cross-entropy on the gold and decomposes weight over candidate trajectories on silver.

**Structural conditioning.** For each cluster with a real synthase we predict the KR+ACP didomain (or full-length protein for divergent PR-PKS the canonical PF08659 HMM does not detect—e.g. citrinin's CitS, whose KR is silent to the conventional profile even at relaxed thresholds; we fold the 2593 aa CitS in full and rescue the anchor by motif scan). For every *(cluster, candidate, cycle)* we execute the cycle prefix deterministically, 3D-embed the ACP-tethered intermediate, anchor its phosphopantetheine sentinel at the ACP-Ser:OG oxygen, rotate the growing chain along the local

ACP-Ser→protein-centroid axis, and extract a five-dimensional pose descriptor: 5 Å contact count partitioned into polar (N/O/S) and hydrophobic (C) subsets, ligand buried fraction, and pocket void volume. These run alongside the substrate-prefix features (cycle index, fractional position, subclass indicator, running chain length and oxidation counts, previous cycle's reduction rank).

**Two engines, one policy.** Three feature regimes on the same $n = 62$ conformational sub-corpus reveal a precise partition (Table 4, Fig. 5A). The partition is governed not by cluster identity but by the *symmetry of the reduction-label sequence across the candidates in* $Z^\star(B)$.

*Substrate-Prefix Engine (label-distinguishable candidates).* When the candidates in $Z^\star(B)$ differ in their per-cycle reduction labels—e.g., isoterrein's three candidates (dh, dh, keto), (dh, keto, dh), (keto, dh, dh) that differ in the cycle index of each DH event, or asperlin's 5 candidates that differ similarly—substrate-prefix features alone route posterior mass cleanly: held-out entropy drops to 0.05 for isoterrein and 0.38 for asperlin (against $\log 3 = 1.10$ and $\log 5 = 1.61$ ceilings). The pose channel adds geometric noise to an architecture the substrate counter has already decoded.

*Geometric Rescue Engine (label-symmetric candidates).* When the candidates share an *identical* reduction-label sequence and differ only in the cycle index of a mass-budget-neutral event—e.g., citrinin's two candidates both reduce to (dh, keto, keto, keto, keto) and differ only in which of cycle 1 or cycle 2 carries the single C-MeT firing—the marginal- likelihood gradient is symmetric under ($\text{cand}_1 \leftrightarrow \text{cand}_2$) and *cannot* distinguish them from any function of substrate prefix alone. Citrinin's held-out entropy under the substrate channel locks at $0.69 = \log 2$ to numerical precision. The pose channel alone barely moves it (0.64); the static cavity geometry without temporal context cannot localize which cycle's geometry it is measuring. The two channels *jointly* drop the entropy to 0.01 with the model committing ∼99.9% probability mass to one specific candidate. This is the regime where the structural channel is not merely helpful but architecturally necessary.

| | Held-out candidate entropy $H_{\text{held}}$ (nats) | | |
|---|---|---|---|
| Silver asset | Substrate-prefix | Pose only | Substrate + pose |
| citrinin ($\lvert Z^\star\rvert$=2, $\log 2$=0.69) | 0.69* | 0.64 | **0.01** |
| isoterrein ($\lvert Z^\star\rvert$=3, $\log 3$=1.10) | **0.05** | 1.07 | 0.50 |
| asperlin ($\lvert Z^\star\rvert$=5, $\log 5$=1.61) | **0.38** | 1.26 | 0.45 |

Table 4: The dual-engine partition. Bold = the resolving regime for each asset. *Citrinin's substrate-prefix entropy locks at exactly $\log 2$ because its two candidates share an identical reduction-label sequence and differ only in C-MeT cycle index; the marginal-likelihood gradient is symmetric under candidate exchange. The pose channel is architecturally necessary precisely where the substrate channel hits this symmetry wall.

**Directional gating: when the in-vivo program is in $Z^\star(B)$.** The two engines establish that the policy commits within $Z^\star(B)$; whether the committed candidate matches in-vivo biology depends on whether the in-vivo program is in $Z^\star(B)$ at all—a question of executor coverage (§13), not of the resolution mechanism. The corpus contains both regimes.

For *isoterrein*, whose biosynthesis proceeds through a linear partially-reducing polyketide before the post-PKS metalloenzyme TerB contracts the chain to the cyclopentenone, the substrate channel routes posterior 0.801 to (dh, dh, keto) hydrolysis—reductions concentrated at the early cycles, matching the canonical HR-PKS architecture—and posterior 0.0001 to the late-reduction (keto, dh, dh) parsimony-best. For *asperlin*, a hexaketide-derived lactone, the substrate channel

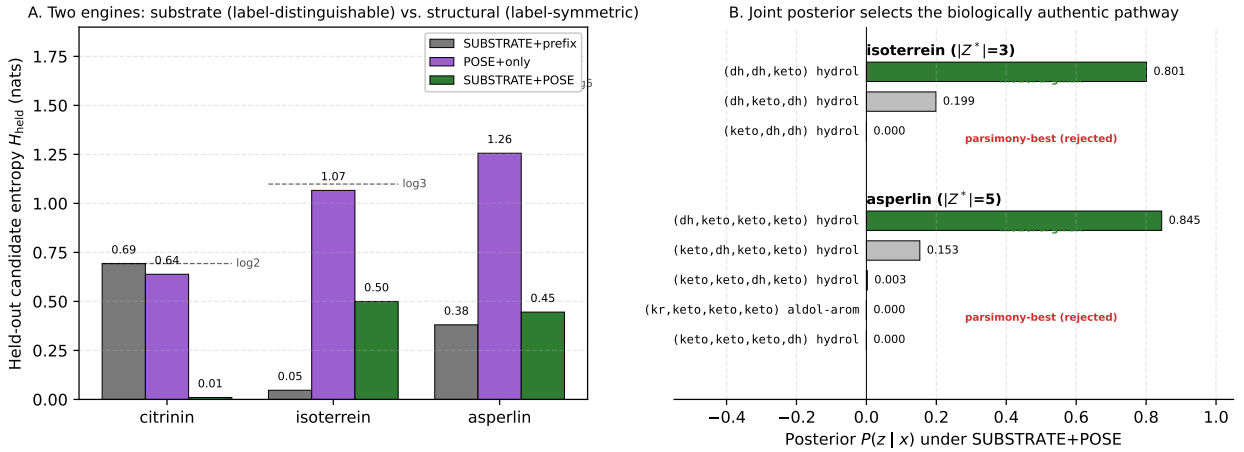


Figure 5: **The dual mechanisms of neuro-symbolic resolution. (A)** Held-out candidate distribution entropy ($H_{\text{held}}$) across three feature regimes. The substrate-prefix channel effortlessly resolves assets with label-distinguishable reduction sequences (isoterrein, asperlin). For label-symmetric candidates sharing identical reduction trajectories (citrinin), the sequence channel encounters a permanent symmetry wall ($\log 2$), which is cleanly shattered ($H_{\text{held}} \to 0.01$) only upon the introduction of 3D pose cavity descriptors. **(B)** Joint posterior probability distributions for isoterrein and asperlin. The multi-channel framework consistently routes mass to the biologically authentic pathway: isoterrein's (dh, dh, keto) early-reduction TerA polyene precursor at posterior 0.801, asperlin's (dh, keto, keto, keto) non-aromatic linear lactone precursor at 0.845. The parsimony-best alternatives—(keto, dh, dh) for isoterrein, the aromatic-release candidate for asperlin—are actively rejected at $\leq 0.001$.

selects (dh, keto, keto, keto) hydrolysis at posterior 0.845 and crucially *rejects* the parsimony-best aromatic-release candidate at posterior ∼0: asperlin is a lactone, not an aromatic, and the substrate-prefix weighting under marginal-likelihood training correctly disfavours the wrong ring topology. Both are directional positives.

For *citrinin*, where the structural-rescue engine commits with posterior 0.999 to a specific candidate, the literature establishes a different fact: heterologous-expression studies of CitS yield a *trimethylated* pentaketide aldehyde requiring *two* C-MeT events (one triketide, one pentaketide) (Storm and Townsend, 2017). Neither of the two candidates in $Z^\star$ at $C_{13}H_{14}O_5$ contains two C-MeT events—the grammar cannot synthesize the in-vivo program at this formula. The entropy collapse demonstrates that the structural channel carries per-cluster discriminating information; the directional gating is the executor-coverage axis our Section 13 roadmap addresses.

**Reading.** The per-cycle policy is empirically a joint function of $G$ and $s_t$, in the form the Outlook predicted, but the decomposition reveals that each channel does specific work. The substrate-prefix engine resolves candidates whose distinct reduction-label sequences leave a non-symmetric gradient signal; the structural-rescue engine resolves candidates whose mass-and-grammar-consistent permutations leave the substrate gradient signal symmetric. The neuro-symbolic architecture compounds: the sound executor bounds $Z^\star(B)$ so a learned policy rules within a verified set rather than over an open structure space, and the appropriate channel routes within that set per the chemistry of the candidates the verifier returns. The result is a policy whose neural component would have nothing principled to commit to absent the executor, and whose symbolic core would

not distinguish label-symmetric candidates absent the structural channel.

## 12 Related work

Riedling et al. (2024) establishes the 51–68% ceiling and the near-flat cross-domain result that motivates this work. Rule-based detectors and product predictors—antiSMASH/fungiSMASH (Blin et al., 2023), PRISM (Skinnider et al., 2020), DeepBGC (Hannigan et al., 2019), GECCO (Carroll et al., 2021), CLOCI (Konkel et al., 2024)—detect clusters and, for bacteria, read off colinear products; none induce an iterative program. ClusterCAD (Eng et al., 2018; Tao et al., 2023) is the modular-PKS intermediate database we both consume (for the cross-taxa transfer probe) and cite as the bacterial existence proof that colinear assembly lines are tabulable (the conceptual result of Section 6). MIBiG (Zdouc et al., 2025) is our pair source; BGC family methods such as BiG-SLiCE (Kautsar et al., 2021) define the within-family neighbourhoods the cross-taxa transfer probe uses. Property/arrangement predictors such as CHAMOIS (CHAMOIS, 2025) infer ontology-level chemistry, not exact iterative programs. Methodologically we draw on execution-guided, neurosymbolic program synthesis (Ellis et al., 2021) and on protein language models (Lin et al., 2023); chemistry is handled with RDKit (Landrum et al., 2025). Our distinguishing stance is to *measure* exact reconstructibility and to ground every claim in a sound deterministic verifier.

## 13 Limitations and outlook

**What this does *not* claim.** We do not claim complete fungal structure prediction, complete NRPS or hybrid coverage, or validation on real untargeted-metabolomics spectra. *Nor a replacement for structure elucidation*: the engine is a metabologenomics *triage* tool—it selects experiments and names the designs where mass/MS-MS triage is insufficient and expression+NMR is required (the path by which all seven verified products in our corpus were pinned)—not a structure-determination method. We claim a grammar-agnostic, verifier-grounded inference *and design* architecture—an inverse compiler with a shared observability and three-state-verdict layer—and demonstrate it on PKS, NRPS, and a typed PKS–NRPS hybrid grammar. The NRPS and hybrid grammars are v0 demonstrations of architecture generality, not full family coverage. Its natural v1 extension is *observation-updated probabilistic realizability*—turning the curated edit tiers into success distributions fit to wet-lab build outcomes, closing the design–build–test loop the planner opens. With that scope fixed once, the specific limitations are:

- **Partial executor coverage.** Single-mode cyclization plus a tetraketide-aromatic and a PT-naphthalene class (skeleton-correct; hydroxyl regiochemistry flagged) are implemented; the remaining PT-aromatic classes, the Diels–Alder, and macrolactonization are not. Thus the 95/99 core-unreachable count is partly executor coverage and partly true rearrangement.
- **Additive-only tailoring.** The $\sim$12 edit types are additive and the joint verifier is "exact-by-budget" (skeleton embedding + $\Delta$-formula + gene/parsimony budget), threshold-free but not yet full atom-by-atom positional reconstruction (positions affect $|Z^\star|$ multiplicity, not existence).
- **Gene budget over-counts** when a locus accession is a whole contig (non-cluster genes included); HMMER on cluster-scoped sequences would tighten it.
- **Subclass unknown for** 304/326; stereochemistry is not modelled (the executor is achiral by design); numbers are over MIBiG-deposited entries, biased toward well-studied (often heavily tailored) compounds.

- **The mining pipeline has one real-cluster case study, not yet a benchmark.** A blind real-cluster, real-mass run (6-MSA, Section 7) verifies the known product at rank 1; beyond it the alphabet+mass+MS/MS chain is shown as near-unique recovery on reachable cores and as fragment-fingerprint *separability* under a crude in-silico fragmenter. A multi-cluster benchmark, real LC-MS/MS validation (e.g. CFM-ID / GNPS), per-cluster domain alphabets from antiSMASH/fungiSMASH, and a learned candidate-ranking prior (data-limited at present) are the natural next steps. In clean form mass and MS/MS act as hard masks; in real metabolomics (adducts, isotopes, noisy or partial spectra) they become high-confidence constraints with explicit uncertainty; we *implement* a matching approximate verifier $Z^{\star}_{\epsilon} = \{\, z : \mathrm{mass}(\mathrm{Exec}(z)) \in [m-\epsilon, m+\epsilon],\ \mathrm{MS/MS}(z) \geq \tau \,\}$ as an adduct-aware, ppm-tolerant, tolerance-scored layer shared across grammars (built via a mass-window formula prefilter so the tolerance is genuine, not an exact verifier with a cosmetic window) alongside the exact one—validation against real spectra (above) being the open step.

Table 5: Executor-coverage roadmap: operator extensions and the product classes they would unlock, turning "partial coverage" from a limitation into a prioritised programme.

| Operator extension | Unlocks | Difficulty / risk |
|---|---|---|
| more small-aromatic closures | many NR/PR aromatics | low |
| PT-domain regiochemistry classes | larger aromatic folds | medium |
| macrolactonization | resorcylic-acid lactones, macrocycles | medium |
| PKS–NRPS handoff checkpoint | hybrid cores | high |
| oxidative skeletal rearrangement | aflatoxin / sterigmatocystin | high (ambiguous) |
| dimerization / depside coupling | non-local products | high (non-local) |

## 14 Conclusion

A fungal BGC is a constrained *program space*, not a molecular blueprint, and a single sound inverse compiler over that space serves three jobs on one spine $y = \mathrm{Exec}_{\Theta}(z; G)$: run *forward* it renders programs to structures; run *inverse* it reconstructs known products and assigns them to metabolome peaks, returning a typed verdict rather than a single accuracy score; run *backward* it designs. The diagnostic half characterizes the natural manifold the engine operates over—exact reconstruction at 8/326 is its *boundary* (not a disappointment), the $1.00 \to 0.567$ cross-taxa collapse its one hard axis, and structural-similarity proxies are shown *not* to be reconstructibility metrics. The contribution is what the backward direction buys: a *realizability geometry* that scores any target molecule by its edit distance to that manifold along two axes—structural (documented domain engineering) and control (the iteration-program frontier)—under a cost model that *is* the field's measured difficulty, curated from primary sources rather than asserted; across the manifold's 1-edit neighbourhood fungal design is mostly iteration-program editing (69 frontier to 49 engineerable), the axis only a sound per-cycle executor can express. An experiment-selection planner then ranks the observation that most collapses the candidate set per unit cost and, crucially, *names its own limit*—the designs where mass and MS/MS triage suffice versus those that must go to expression+NMR. The same iteration-grammar wall is therefore a prediction ceiling read one way and a design frontier read the other: one measurement, a limitation and the contribution at once. Formally these are projections of one observability quantity $\kappa_O$ on one frontier $\mathcal{F}_O$ (Theorem 1), so the same engine, the same planner, and the same experiments serve both directions—and the same theorem predicts the

architecture reproduces its results on any family meeting the soundness conditions of §4.1. We do not replace structure elucidation; we build the missing middle—the sound, cost-weighted layer between target-blind generative chemistry, enzyme-blind retrosynthesis, and undirected genome mining—on which other models compose without eroding soundness: a sequence policy reranking $Z^\star$ and, as the natural next composition, a bioactivity predictor filtering it, yielding *target-directed* natural-product discovery—a search for the cheapest cluster-plus-edits route to a desired activity, each proposal arriving with a construction plan and a confirming experiment. *We do not claim that pipeline here*; we claim the substrate on which it becomes possible. The tools, benchmark, and analyses are open and reproducible.

**Near-term extensions.** The architecture is grammar-agnostic—observability and the three-state verdict are shared across PKS, NRPS, and a typed hybrid—so deepening the family grammars (NRPS substrate specificity, the tetramic-acid release) and extending executor coverage widen the manifold without touching the shared layers. A learned, sequence-aware *policy* fits the engine naturally as a soundness-preserving *reranker* over the legal program set, and the controlled per-step/pooled comparison of §6.4 predicts where it pays off: on *bacterial modular* PKS, where each module carries its own sequence (a companion demonstration), not on the fungal iterative wall, which is structural by construction—for fungi the policy ranks within $Z^\star$, it does not break the wall. Two further steps close the loop the planner opens: validating the observability layer on real LC-MS/MS spectra (e.g. CFM-ID / GNPS), and turning the curated edit tiers into *observation-updated probabilistic realizability*—success distributions fit to wet-lab build outcomes—so the design–build–test cycle, not only experiment selection, runs on the same spine. The reference implementation (`lpi.engine`) is released alongside.

# 15 Outlook: the iteration program as a partially observed dynamical control problem

These results invite a reframing of the fungal problem itself. The per-cycle action may not be a function of the synthase alone—$a_t \sim P(a_t \mid G)$—but of the synthase *and* an evolving substrate state, $a_t \sim P(a_t \mid G, s_t)$, where $s_t$ is the chain length, oxidation state, carrier-protein pose, and transient conformation the constant catalytic machinery presents to itself on cycle $t$. Bacterial modular PKS *externalize* this state into architecture—one module per step—whereas the fungal iterative synthase *compresses* the entire policy into one reusable catalytic manifold with state-dependent routing: a non-equilibrium, trajectory-dependent object closer to a substrate-conditioned automaton than to a static sequence→structure map. Crucially this is an *architectural* claim, not a statistical null: an iterative synthase exposes no per-step observable by construction, and where one *does* exist—bacterial modular PKS—the same predictor under the same executor nearly doubles ($0.50 \to 0.78$, §6.4), so the model class is not the limit. The fungal probes then *corroborate* the boundary by elimination: the policy is recovered neither from chain position nor from the computable running substrate state, so the identifying signal lies in the conformational component of $s_t$ that a synthase-local view cannot, by construction, supply. This decomposes the problem into three layers: the sound executor (built here—and what makes the rest *identifiable*, by bounding the combinatorics a learned model would otherwise drown in), a per-cycle policy $\pi_\theta(s_t)$, and a state-evolution model $s_{t+1} = F(s_t, a_t)$ that structural-dynamics observables (MD ensembles, cryo-EM states, chain-length-conditioned docking) would supply. The requirement this hands such an effort is sharp: the policy cannot be learned from synthase-local static observables, so the next advance needs either a substantially larger corpus read at per-cycle resolution or direct partial observables of the conformational state.

The first measurable breakthrough need not be complete product prediction—it would be a single per-cycle decision becoming predictable above 0.567 once cycle-distinguishing state is injected, which the executor would then propagate, soundly, through the rest of the program. We take a first empirical step along this axis in §11: AlphaFold2-conditioned pocket descriptors injected alongside the substrate-prefix index resolve the label-symmetric C-methylation degeneracy in citrinin from $H = \log 2$ to $H = 0.01$ on held-out evaluation, with a complementary substrate channel cleanly resolving label-distinguishable cases (isoterrein, asperlin) on the same corpus. The two engines do specific work: the substrate counter localizes which cycle is being asked about, the structural channel routes within that cycle's pocket-substrate geometry, and the executor bounds the set within which either can rule.

# A Reproducibility

Python 3.12, RDKit 2025.9.6; optional `torch`/`transformers` for the cross-taxa transfer and ESM-2 components. Each result maps to one command: `make data` (the 326 pairs), `make roundtrip` (executor gate), `make search` (the verifier and $Z^\star$), `make reachability` (exact-match 8/326), `make corematch` (the core-match sensitivity), `make phase0c` (tailoring-latent 7/106), `make clustercad` and `make differential` (the cross-taxa transfer probe), `make crossgrammar`, `make observability`, and `make realdemo` (the cross-grammar PKS/NRPS/hybrid demo, the candidate-collapse observability benchmark, and the blind real-cluster 6-MSA mining demo), `make figures`, and `make test` (161 tests); the realizability geometry and experiment-selection planner are reproduced by `scripts/worked_design_6msa.py` and `scripts/planner_tree_figure.py` (Figures 3–4); the per-step vs. pooled control behind the structural-wall claim (§6.4) by `scripts/explorations/bacterial_control.py`. The tagged milestones (`lpi-v0.2-cross-grammar` through `lpi-v0.17-homology-clean`, including `v0.14-curated-cost-basis`, `v0.15-planner`, `v0.16-sequence-head`, and `v0.17-homology-clean`), per-result numbers, scripts, and commits are tabulated in the repository `README` and `PAPER_READINESS.md`.

# References


O. Riedling, A. K. Walker, et al. Predicting fungal secondary metabolite activity from biosynthetic gene cluster data using machine learning. *Microbiology Spectrum*, 2024.

K. Blin et al. antiSMASH 7.0. *Nucleic Acids Research*, 2023.

M. A. Skinnider et al. Comprehensive prediction of secondary metabolite structure and biological activity (PRISM 4). *Nature Communications*, 2020.

C. H. Eng, T. W. H. Backman, et al. ClusterCAD: a computational platform for type I modular polyketide synthase design. *Nucleic Acids Research*, 46(D1):D509–D515, 2018.

X. B. Tao, S. LaFrance, Y. Xing, A. A. Nava, H. Garcia Martin, J. D. Keasling, and T. W. H. Backman. ClusterCAD 2.0: an updated computational platform for chimeric type I polyketide synthase and nonribosomal peptide synthetase design. *Nucleic Acids Research*, 51(D1):D532–D538, 2023.

G. D. Hannigan et al. A deep learning genome-mining strategy for biosynthetic gene cluster prediction (DeepBGC). *Nucleic Acids Research*, 47(18):e110, 2019.

L. M. Carroll et al. Accurate de novo identification of biosynthetic gene clusters with GECCO. *bioRxiv*, 2021.

Z. Konkel, L. Kubatko, and J. C. Slot. CLOCI: unveiling cryptic fungal gene clusters with generalized detection. *Nucleic Acids Research*, 52(16):e75, 2024.

M. M. Zdouc, K. Blin, et al. MIBiG 4.0: advancing biosynthetic gene cluster curation through global collaboration. *Nucleic Acids Research*, 53(D1):D678–D690, 2025.

S. A. Kautsar, J. J. J. van der Hooft, D. de Ridder, and M. H. Medema. BiG-SLiCE: a highly scalable tool maps the diversity of 1.2 million biosynthetic gene clusters. *GigaScience*, 10(1):giaa154, 2021.

Machine learning inference of natural product chemistry across biosynthetic gene cluster types (CHAMOIS). *bioRxiv*, 2025. `https://doi.org/10.1101/2025.03.13.642868`.

S. M. Ma, J. W.-H. Li, J. C. Vederas, Y. Tang, et al. Complete reconstitution of a highly reducing iterative PKS (lovastatin nonaketide synthase LovB/LovC). *Science*, 326(5952):589–592, 2009.

J. M. Crawford et al. Structural basis for biosynthetic programming of fungal aromatic polyketide cyclization (the product-template domain). *Nature*, 461:1139–1143, 2009.

K. Ellis et al. DreamCoder: bootstrapping inductive program synthesis with wake-sleep library learning. *PLDI*, 2021.

Z. Lin et al. Evolutionary-scale prediction of atomic-level protein structure with a language model (ESM-2/ESMFold). *Science*, 379(6637):1123–1130, 2023.

G. Landrum et al. RDKit: open-source cheminformatics (version 2025.09.6), 2025. `https://www.rdkit.org`.

R. J. Cox. Curiouser and curiouser: progress in understanding the programming of iterative highly-reducing polyketide synthases. *Nat. Prod. Rep.*, 40:9–27, 2023. doi:10.1039/D2NP00007E.

K. M. Fisch, W. Bakeer, A. A. Yakasai, Z. Song, J. Pedrick, Z. Wasil, A. M. Bailey, C. M. Lazarus, T. J. Simpson, R. J. Cox. Rational domain swaps decipher programming in fungal highly-reducing polyketide synthases and resurrect an extinct metabolite. *J. Am. Chem. Soc.*, 133:16635–16641, 2011. doi:10.1021/ja206914q.

E. M. Musiol-Kroll and W. Wohlleben. Acyltransferases as tools for polyketide synthase engineering. *Antibiotics (Basel)*, 7(3):62, 2018. doi:10.3390/antibiotics7030062.

J. R. Jacobsen, C. R. Hutchinson, D. E. Cane, and C. Khosla. Precursor-directed biosynthesis of erythromycin analogs by an engineered polyketide synthase. *Science*, 277(5324):367–369, 1997. doi:10.1126/science.277.5324.367.

L. Kellenberger, I. S. Galloway, G. Sauter, G. Böhm, U. Hanefeld, J. Cortés, J. Staunton, and P. F. Leadlay. A polylinker approach to reductive loop swaps in modular polyketide synthases. *ChemBioChem*, 9(16):2740–2749, 2008. doi:10.1002/cbic.200800332.

S. Yuzawa, T. W. H. Backman, J. D. Keasling, and L. Katz. Synthetic biology of polyketide synthases. *J. Ind. Microbiol. Biotechnol.*, 45(7), 2018.

L. M. Repka, J. R. Chekan, S. K. Nair, and W. A. van der Donk. Mechanistic understanding of lanthipeptide biosynthetic enzymes. *Chem. Rev.*, 117(8):5457–5520, 2017. doi:10.1021/acs.chemrev.6b00591.

M. S. Donia, J. Ravel, and E. W. Schmidt. A global assembly line for cyanobactins. *Nat. Chem. Biol.*, 4:341–343, 2008. doi:10.1038/nchembio.84.

P. M. Storm and C. A. Townsend. Functional and structural analysis of programmed C-methylation in the biosynthesis of the fungal polyketide citrinin. *Cell Chemical Biology*, 24(3):316–325, 2017. doi:10.1016/j.chembiol.2017.02.005.